

Surname

Name

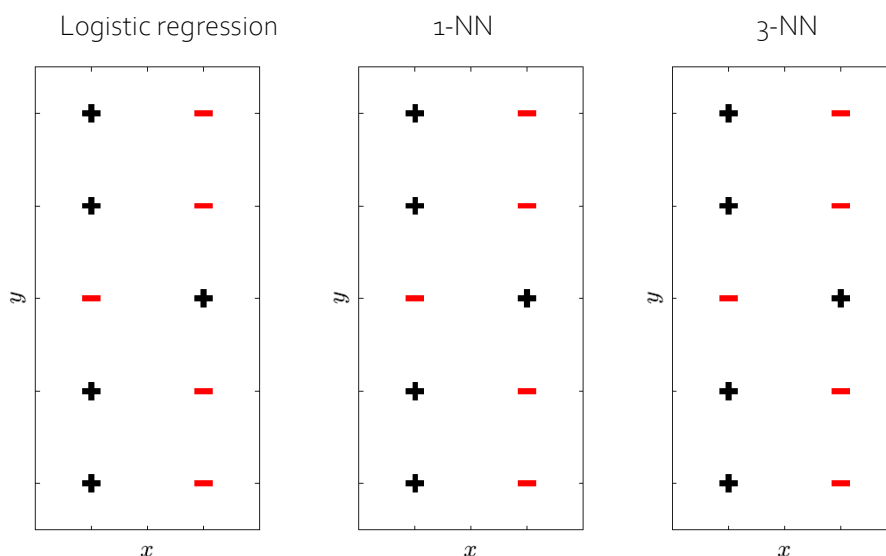
University ID Number

Signature

- =====
- Write the solutions in the blank areas (use the back of the page, if needed)
 - The number of pages is 4. Additional papers will not be considered.
 - Clarity, order and precision are strongly considered for the final evaluation.
- =====

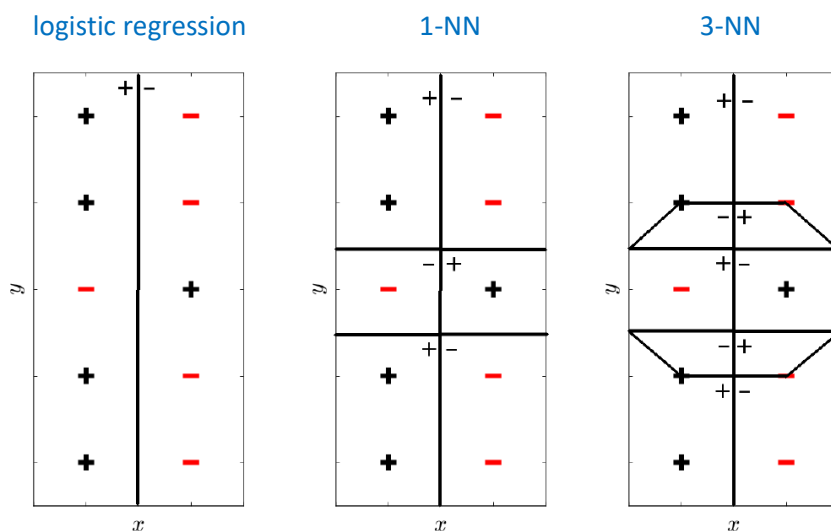
1. [Classification]

On the 2D dataset below, draw the decision boundaries learned by the following algorithms (where NN stands for “Nearest Neighbor(s)”). *Be sure to mark which regions are labeled positive or negative.*



Then, illustrate the main pros and cons of each algorithm and describe when to prefer one over another.

Solutions



For the second part of the question, see the lecture notes (chapters 02 and 06).

2. [Neural networks]

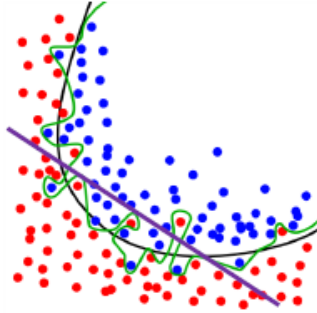
- a. Provide an example of nonlinear classification problem using a multi-layer perceptron.
- b. Does increasing the number of hidden nodes in a multilayer perceptron improve generalization? Why (not)?
- c. If gradient descent is used for weight optimization in a feedforward neural network, overfitting is not possible, as gradient descent is guaranteed to find a local optimum. Say whether the previous statement is true or not (and why).
- d. When training neural networks, the network weights are updated more often in stochastic gradient descent or in standard gradient descent?

Solutions

- a. Any case of a target function given by a logical operation including both AND and OR operators. See, e.g., the example given in the lecture notes (chapter 09, page 3).
- b. Short answer: increasing the number of hidden nodes increases its representational ability, which increases the risk of overtraining (learning all patterns by heart in the extreme case). Longer answer: even if the previous statement is generally true, if the initial number of hidden nodes was too small to solve the problem at all, then increasing the number should improve generalization (up to the point where the number of nodes reaches the required number to represent the function).
- c. The statement is false. Local or global optimum has nothing to do with overfitting.
- d. In stochastic gradient descent, as updates occur after every pattern/sample presentation. The standard approach only accumulates the weight changes until the whole batch of samples has been presented once.

3. [Overfitting]

- a. What do we mean by “overfitting”, and why can it become a problem? You may refer to the figure below in the discussion or make examples.



- b. What do we mean by “bias”? Again, you may refer to the figure in the discussion or make examples.
- c. You are facing a classification problem. You first try a perceptron model. What are the possible problems you may encounter with “overfitting” and “bias”?
- d. You want to address the bias problem. You still want to use a perceptron as a learning model. Which other settings can you change to get a model which faces less problems with the bias, and how will you change them? Which new problems may these improvements cause?
- e. You are training a logistic regression classifier. Which problems can arise with respect to overfitting and bias?
- f. Describe one method for avoiding overfitting.

Solutions

- a. Whenever we choose a random sample of individuals from a larger population, there might be attributes that have a different distribution in the sample than in the population at large. If the learning algorithm focus too much on these data, it may not fit other samples from the population equally well. The wavy line in the figure could be an example of this (see also the lecture notes, chapter 08, for further details).
- b. By choosing a learner, we can only train models that are within the class of the learner, e.g., a perceptron can only produce a linear classifier. The decision border of any linear classifier will be a straight line, like the purple line in the figure. We see that this is not a good classifier for this problem.
- c. A perceptron can only result in a linear model corresponding to a straight line. This is a strong bias. Many problems are, however, nonlinear. A linear model can also experience problems with overfitting if there are too many features.
- d. You can engineer your features, e.g., by adding polynomial forms of them. The danger is that if you are too clever in the engineering, you may overfit the data. For example, if you have more features than data, you may train a perfect model on the training data, which does not generalize very well.
- e. The logistic regression classifier can also only predict linear decision boundaries and run into similar problems as the perceptron with respect to bias and overfitting.
- f. To avoid overfitting, one possibility is to use L2-regularization, which punishes large weights. Thereby, it avoids putting too much emphasis on some features which may have a distribution in the training data very different from its distribution in the population at large. See the lecture notes, chapter 08, for further details.

4. [Unsupervised learning]

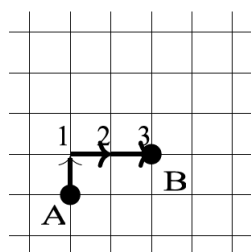
Alice and Bob, two colleagues from the astrophysics department, have given you a collection of astronomical data describing exoplanets in different star systems. Each exoplanet is described by the distance from its orbiting star in AU (astronomical units), its mass (as multiples of the Earth), and the degree of light reflection (as an *albedo* integer).

	AU from star	Mass	Albedo
HD 209458 b	2	3	7
HD 189733 b	5	3	3
51 Pegasi b	7	2	5
PSR B1257+12 B	3	5	6
PSR B1257+12 C	5	4	5
OGLE-TR-56 b	7	4	3
Fomalhaut b	3	3	8
2M1207 b	4	3	7

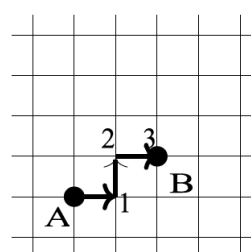
Some of these data (about half) have been collected using the transit detection method and others using an infrared detection method. Alice and Bob know that these two methods are sensitive to exoplanets with different features, but they do not know which sample has been collected with which method.

Alice argues that looking at *AU from the star* and *Albedo* may help them infer which observations were performed with which techniques; Bob holds that looking at *AU from the star* and *Mass* may provide a better perspective to group the exoplanets by their discovery method.

- Plot the data first according to Alice's hypothesis and then Bob's hypothesis. Which hypothesis seems more likely?
- To give more grounding to your conclusions, you decide to run the **k-means** algorithm on your data using two clusters, one for each detection method. Let us call one cluster the blue cluster, and the other one the red cluster. Run three iterations of k-means (assignment, recomputation of the centroids) on Alice's and Bob's data. Initialize the blue cluster at (3,2), and the red cluster at (8,4). In the assignment phase, use as a distance function the *Manhattan distance* $D_{Man}[x_i, x_j]$, that gives you the number of straight segments necessary to get from one point to the other:



Manhattan distance



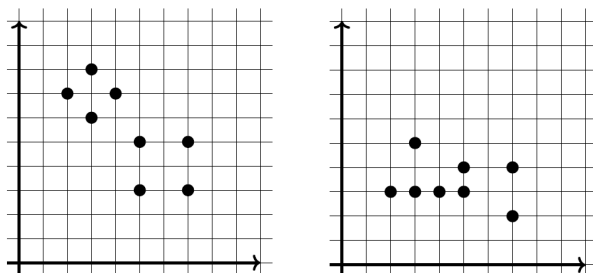
Notice that the distance is independent from the path.

If a point is the same distance from the center of both clusters, assign it to the blue cluster. In the recomputation of the centroids, round the values of the nearest integer ($5.3 \rightarrow 5$, $8.7 \rightarrow 9$).

How do your results agree with your conclusions from the visualization exercise at the previous point (a)?

Solutions

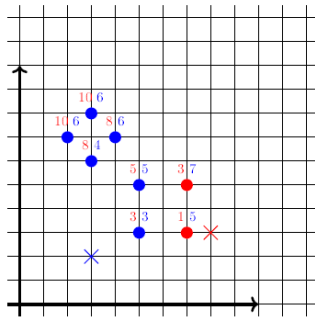
- Alice (left) and Bob (right):



Alice's hypothesis seems to highlight more structure in the data.

b. Alice's iterations

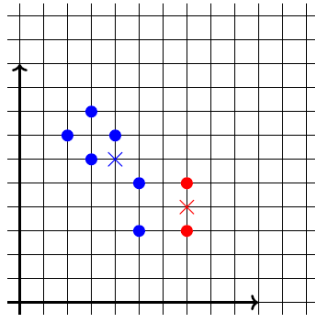
Alice's data - Iteration 1 - Assignment:



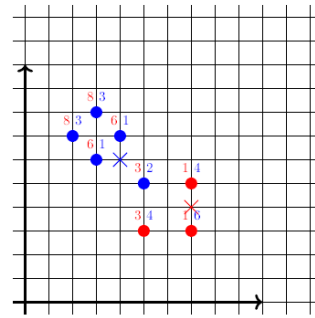
Alice's data - Iteration 1 - Recomputation:

$$x = 2 + 3 + 3 + 4 + 5 + 5 = 22/6 = 4; y = 3 + 5 + 6 + 7 + 7 + 8 = 36/6 = 6$$

$$x = 7 + 7 = 14/2 = 7; y = 3 + 5 = 8/2 = 4$$



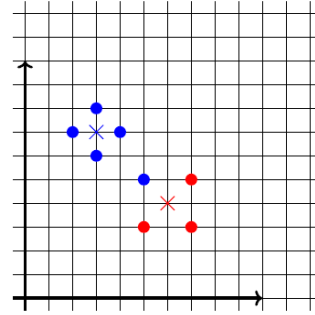
Alice's data - Iteration 2 - Assignment:



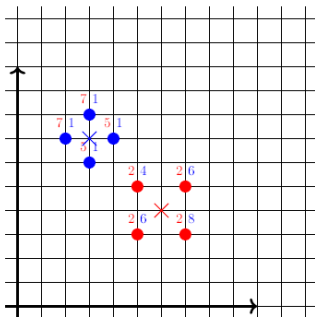
Alice's data - Iteration 2 - Recomputation:

$$x = 2 + 3 + 3 + 4 + 5 = 17/6 = 3; y = 5 + 6 + 7 + 7 + 8 = 33/5 = 7$$

$$x = 5 + 7 + 7 = 19/3 = 6; y = 3 + 3 + 5 = 11/3 = 4$$



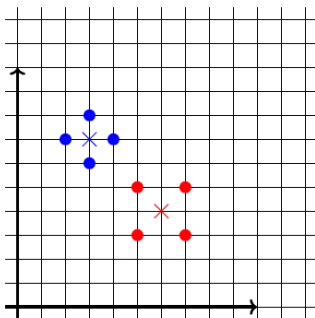
Alice's data - Iteration 3 - Assignment:



Alice's data - Iteration 3 - Recomputation:

$$x = 2 + 3 + 3 + 4 = 12/4 = 3; y = 6 + 7 + 7 + 8 = 28/4 = 7$$

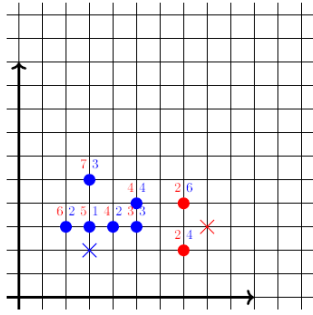
$$x = 5 + 5 + 7 + 7 = 24/4 = 6; y = 3 + 3 + 5 + 5 = 16/4 = 4$$



Bob's iterations

Solution:

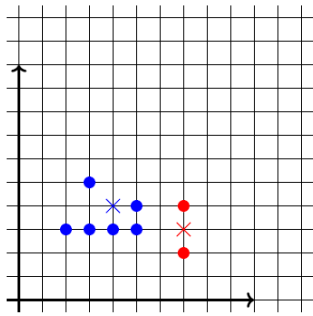
Bob's data - Iteration 1 - Assignment:



Bob's data - Iteration 1 - Recomputation:

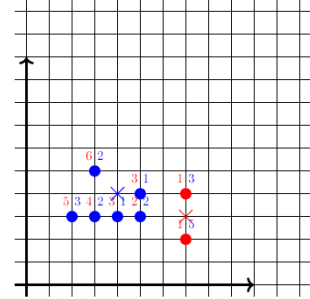
$$x = 2 + 3 + 3 + 4 + 5 + 5 = 22/6 = 4; y = 3 + 3 + 3 + 3 + 4 + 5 = 21/6 = 4$$

$$x = 7 + 7 = 14/2 = 7; y = 2 + 4 = 6/2 = 3$$



Solution:

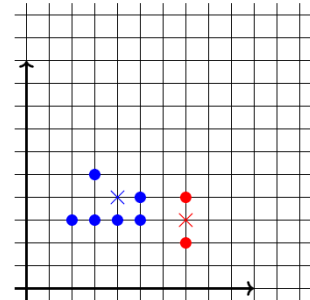
Bob's data - Iteration 2 - Assignment:



Bob's data - Iteration 2 - Recomputation:

$$x = 2 + 3 + 3 + 4 + 5 + 5 = 22/6 = 4; y = 3 + 3 + 3 + 3 + 4 + 5 = 21/6 = 4$$

$$x = 7 + 7 = 14/2 = 7; y = 2 + 4 = 6/2 = 3$$



Iteration 3 is the same. Nothing changed.

The k-means algorithm confirms that Alice's hypothesis allows the two clusters to be clearly distinguished.